

Trustworthy LLM-Assisted Detection of Malicious Packages: Cross-Language Behavior Graphs, Federated Learning, and the Safety Layer as a Third Threat

Anonymous (double-blind submission)

Anonymous

ABSTRACT

We present a pilot study that unifies three threat surfaces against LLM-assisted malicious-package detection: (i) the detection model itself, trained over a normalized cross-language *behavior-graph* representation; (ii) the training regime, a federated simulation over naturally non-IID ecosystem partitions (npm vs. PyPI); and (iii) the safety layer of the LLM used for analysis, which may refuse, or corrupt verdicts of, defensive package reviews. We release a fully provenance-tracked corpus of 603real npm/PyPI packages (390 malicious from a wild-capture dataset, 213 popularity-derived benign), extract six-class behavior graphs with tree-sitter, and train logistic/MLP models under FedAvg/FedProx with secure-aggregation and DP simulations, under a registered twenty-seed protocol (AMENDMENT-4), then re-validated under a pre-registered *corrected baseline* (AMENDMENT-5: a properly converged, standardized centralized solver; a stateless hashing text featurizer). Under a package-level group split (our primary protocol, adopted after an audit quantified 35% version-level train-test leakage under random splits), FedAvg is statistically *equivalent* to the strong centralized baseline at the pre-registered TOST margin of ± 0.02 F1 (graph: $\Delta F1 = -.0065$, 90% CI $[-.0133, +.0003]$ — equivalence passes; the text block passes as well) — federation carries no measurable cost where the inference is leakage-resistant; a small centralized advantage appears only on the leakage-prone secondary random split ($\Delta F1 = -.0160$, Holm $p=.025$). An earlier claim that text features degrade under federation *does not survive* this corrected baseline — it was an artifact of a pooled-vocabulary featurizer plus an undertrained solver — and is retracted: under the corrected protocol, text and graph features are both federation-robust. A refusal-gated LLM-built knowledge base — now covering the full corpus API universe (137/137 types, 2,299/2,299 instances) — adds $\approx +1.0$ F1 to centralized training (reproduced in magnitude by the KB-v3 pure-join re-run at the shared seed: $+1.0/+1.1$ points on group/random, graph features), but its FedAvg effect is of mixed sign (-0.1 to -2.2 points; single-seed deltas below seed variability), so its robust yield is gated per-call explanation, not accuracy. In safety-arm generations over real packages — 360 over 60 in the registered batch, 600 over 100 in a follow-up expansion — no overt refusal occurs in any arm (RR $\equiv 0.000$: structural blocking-absence at this scale; consistent with 2,700 measured defensive-task generations across both project domains at 2-3B), so defensive-refusal blocking does not replicate in the package domain; instead, untrusted in-package content (an advisory comment) shifts verdict sensitivity on genuinely malicious packages in a model-dependent direction (malicious recall $.53 \rightarrow .80$ for granite-3.3-2B vs. $.13 \rightarrow .03$ for Llama-3.2-3B)

with zero false positives on benign packages (0/180) — a recall-perturbing channel, unlike the false-positive channel observed in the vulnerability domain; a post-hoc granite-3.3-2B re-run of the vulnerability-domain ladder now shows the reassertion harm replicates at 2B (malicious recall $.76 \rightarrow .07$, $p \approx 4 \times 10^{-10}$) with a benign verdict false-positive bias ($0 \rightarrow 20/30$, $p \approx 2 \times 10^{-6}$) — same model, opposite domain signature. An initial attack-defense pairing asks whether input sanitization can neutralize this channel: an AST-gated comment/docstring strip moves pooled malicious recall from $.438$ (attack) to $.340$ (defended; exact McNemar $p = .219$) — a descriptive shift that hides opposite per-model outcomes: on granite-3.3-2B the strip fully neutralizes the channel (all four attack-induced flips revert to the neutral baseline; benign FP 0), while on Llama-3.2-3B it restores nothing — the four detections it loses are byte-identical snippets of a single package family whose own comments the defense also strips, a defense cost we pre-registered as a possible outcome. We report all negative and null results, including that an earlier apparent FedAvg minority-client harm disappeared once a file-selection bug was fixed, that two further earlier-protocol findings (text degradation under FL; a centralized-training collapse) were re-attributed to baseline defects — pooled-vocabulary fitting and solver undertraining — once a corrected baseline was registered, and that the per-seed significance layer initially predated the refreshed twenty-seed grid and had to be fully recomputed (regeneration now scripted and verified); we discuss the distance between this pilot and a Q1-grade system.

1 INTRODUCTION

Open-source package ecosystems are a primary supply-chain attack surface: attackers publish packages that steal credentials, exfiltrate data, or download further malware during installation [3, 7, 8]. Deployed detectors such as DONAPI [6], Cerebro [10], and MalGuard [4] learn from package metadata, installation behavior, or code; LLM-based triage is emerging independently [9].

This paper asks whether *trustworthy* LLM-assisted detection can be studied as one system rather than three separate papers. We identify three threats that act on the same analysis pipeline:

- (1) **Model threat:** the detector must generalize across languages (JavaScript/Python) and ecosystems from limited labeled data;
- (2) **Training-regime threat:** ecosystem data is naturally non-IID and cannot always be pooled (organizational/privacy boundaries), motivating federated training whose cost must be quantified;
- (3) **Safety-layer threat:** the LLM performing (or explaining) the analysis may *refuse* defensive work phrased

offensively, or *corrupt* verdicts under untrusted package content — findings from a prior vulnerability-domain measurement study [1, 2] motivate testing whether such interference appears in the higher-stakes package domain, where refusal rates are plausibly higher (we measure, not assume; the measured answer at 2–3B is: no refusals, and a model-dependent recall-perturbing channel instead, §5.2).

Our contributions: (1) a provenance-complete 603-sample npm+PyPI corpus with per-sample source, label provenance, and checksums (§4); (2) a normalized cross-language behavior-graph schema with six semantic classes and 18 graph-level features (§3); (3) a federated simulation with secure-aggregation and DP mechanisms over ecosystem partitions, evaluated with a leakage-resistant group split, a registered twenty-seed protocol, and a pre-registered corrected-baseline equivalence test — TOST at ± 0.02 F1: FedAvg is equivalent to a strong centralized baseline on the primary split, with a small centralized advantage confined to the leakage-prone secondary split (§5); (4) a refusal-gated LLM-augmented knowledge base whose measured value is explainability yield — gated per-call justifications covering the full corpus API universe (137/137 types, 2,299/2,299 instances) — with neutral-to-small F1 effect (-0.1 pt FedAvg, $+1.0$ pt centralized); and (5) a pre-registered safety-arm measurement ported from the vulnerability domain to real packages ($n=60$, two $<4B$ models, 360 generations): refusals are absent ($RR = 0.000$) and untrusted in-package content acts as a model-dependent *recall-perturbing* channel on malicious packages with zero benign false positives (§5.2). All audits, amendments, and failed hypotheses are disclosed.

2 RELATED WORK

Malicious package detection. DONAPI maps installation-time behavior sequences for npm [6]; Cerebro trains a single cross-ecosystem model for npm and PyPI (centralized) [10]; MalGuard targets real-time PyPI detection [4] (venue per USENIX program pages; arXiv record verified); MeMPtec uses metadata [5]. None study federated training across ecosystem partitions or the safety layer of LLM analyzers. **FL for security.** VulFL applies FL to vulnerability detection [11] (preprint); to our knowledge (as of September 2026), FL over ecosystem-partitioned clients for malicious-package detection is unexplored — a literature-check hedge, not an exhaustive claim. **LLMs for supply-chain security.** LLM-based npm triage [9] measures LLM detection accuracy but not refusal interference. **Attack taxonomy.** Ladisa et al. [7] systematize OSS supply-chain attacks (IEEE S&P 2023; an earlier draft of this work mistakenly cited ACSAC — corrected). **Safety-layer line.** A prior vulnerability-domain measurement study [1] established that open 2–7B models show $RR \approx 0$ under offensive-but-defensive wording, verdict corruption toward vulnerability (a false-positive channel), and defense bundles that harm $\leq 3B$ models; concurrent work reports a defensive-refusal bias in cyber-defense settings [2]. The present study ports those conditions to packages and measures whether the same channel types appear; at 2–3B they do not: refusals stay at zero and the corruption channel re-types as recall-perturbing (§5.2).

3 METHOD

3.1 Behavior graphs

tree-sitter parsers for JavaScript and Python produce, per sample, a merged graph whose nodes are classified API calls in six semantics (FILE_IO, NETWORK, PROCESS, CRYPTO, DYNAMIC_CODE, DATA_ACCESS) and whose edges are in-scope sequence edges and data edges from produced-variable tracking; imports and require/import aliases are resolved. Entry kinds mark `setup.py` and `npm install-hook` scripts. The schema is versioned (SCHEMA_VERSION 1.1.0). Eighteen graph-level features (class histogram, size/density/depth, scope and import diversity, entry flags, parse-failure count) feed classical models. A GNN over the same graphs is a specified extension (project roadmap, to be pre-registered before any run); this paper reports no GNN results.

3.2 Federated simulation

Clients are ecosystem partitions (npm $n=328$, PyPI $n=146$ after the train split; malicious rate 0.686 vs. 0.527 — naturally non-IID but mildly so). We train LR (primary) and a 32-unit MLP for 15 rounds (2 local epochs, SGD lr 0.1, batch 32, seed 20260922) with FedAvg and FedProx ($\mu=0.01$). Secure aggregation is a pairwise additive-mask *simulation* (masks cancel exactly for two clients); DP is a DP-FedAvg-style clip-and-noise *simulation* with a single-round analytic ϵ (no composition proof; disclosed). Baselines: centralized pooling and per-client-only.

Corrected baseline (AMENDMENT-5, registered before any run). An audit found two baseline defects in the protocol above: the centralized solver (SGD, lr 0.1) does not reliably converge on this feature scale, and the TF-IDF arm fits its vocabulary on the pooled corpus — a centralized preprocessing step that is unavailable under federation. AMENDMENT-5 pre-registers the correction used for all federation claims here: *strong centralized* = logistic regression (lbfgs, max_iter 5000, tol 10^{-6}) on train-fit standardized features, with $C \in \{.01, .1, 1, 10\}$ selected by stratified 3-fold CV on training rows only; *FedAvg* = converged local fit of the same objective on each client’s standardized share, averaged with client-size weights — a valid FedAvg simulation for convex objectives; the round loop still runs and the aggregate is a fixed point from round 2 in all cells (disclosed); *FedProx* = the exact local objective (mean log-loss + $L_2 + \frac{\mu}{2} \|\theta - \theta_g\|^2$) solved by L-BFGS-B with analytic gradients, with μ routed per algorithm (a routing bug that made both earlier arms silently run FedProx is fixed, unit-pinned, and reproducible via a legacy flag; §6); and the *text arm* becomes a stateless HashingVectorizer (2^{18} features, L_2 -normalized) — no vocabulary is fitted anywhere, so the featurizer is legal under federation. Clients share the train-fit scaler and the CV-selected C (a central hyperparameter-coordination assumption; training rows only — no test row is ever touched by any fit).

3.3 LLM-augmented knowledge base

Unknown API calls (from the graph universe, not the feature table) are classified by a local Qwen2.5-Coder-3B-Instruct into the six-class taxonomy with risk levels, refusal-gated (refusal $\rightarrow$ one retry $\rightarrow$ UNSURE; a refusal is never converted into a class) and cached;

KB-derived features (risk ratio, confidence, unsure ratio) augment the classifier for an on/off ablation.

3.4 Safety port

Three pre-registered arms over real package code (60 packages: 30 malicious + 30 benign, selection seed 20260922; the 10 initial-pilot packages kept verbatim, 50 added with zero overlap): **P0** neutral defensive task; **P1** identical intent with defensive-refusal-bias vocabulary [2] (“steal credentials”, “exfiltrate”, “malware”); **P2** an advisory-style security comment injected into the package file (comment-only, gated by a tree-sitter AST-equivalence check). Metrics (metrics_version 2, renamed after a follow-up audit found that the initial field labelled “false-positive bias” actually computed the true-positive rate on *malicious* packages; values unchanged, semantics corrected): refusal rate (RR) per arm; verdict flips benign→malicious and malicious→benign; *malicious recall* (TP-rate on malicious packages) and, separately, the benign false-positive rate; per-model refusal thresholds; frozen decision rules (blocking: $\Delta RR \geq 0.10$ and exact McNemar $p < 0.05$; corruption: ≥ 1 benign→malicious flip).

4 EXPERIMENTAL SETUP

Corpus (603, disclosed). Malicious: DataDog wild-capture dataset, deterministic stride sample (seed 20260922) of 298 npm + 92 PyPI archives with verbatim per-sample class provenance; benign: popularity-ranked npm registry and PyPI download lists, 123 + 90, “assumed benign, not individually audited”. An internal audit found the manifest builder silently skipped DataDog’s flat layout; the fix adds 6 archives (5 single-version packages + 1 extra archive of an already-selected package), 597 → 603; the flawed 597 manifest is retained as an audit artifact. **Coverage.** With-graph rates after the selection fix: npm-benign 105/123 (85.4%), npm-malicious 252/298 (84.6%), pypi-benign 63/90 (70.0%), pypi-malicious 80/92 (87.0%); 500/603 (82.9%) overall. Empty-graph rows are kept as all-zero and reported per cell. **Splits.** PRIMARY: package-level group split (all versions of a package on one side; label-stratified; the original seed instance: test $n=129$, 88 malicious; the split is re-drawn per registered seed). This was adopted as AMENDMENT-1 after measuring that the pre-registered random split shares a package between train and test for 35% of test rows. SECONDARY: the original stratified random split, reported for continuity. **Statistics.** Registered twenty-seed protocol (AMENDMENT-4; seeds 20260922–20260941, extending the earlier five-seed registered grid — the five original seeds are unchanged and seed 20260922 reproduces the original single-seed table to four decimals): per-seed exact McNemar (accuracy@0.5) on the paired FedAvg-vs-centralized comparison, aggregated by two-sided exact Wilcoxon signed-rank over the twenty per-seed $\Delta F1$ (AUC secondary), Holm-corrected over the four registered comparisons. At $n=20$ the minimum attainable two-sided exact p is $2/2^{20} \approx 1.9 \times 10^{-6} < .05$: seed-level tests can reach α . Single-seed comparisons additionally report McNemar (exact when discordant < 25) and paired bootstrap (10k, seed 20260922) on per-sample accuracy. Standard deviations over seeds are population ($ddof=0$). AMENDMENT-4 was registered before the 20-seed run but after the 5-seed results were known (disclosed; not double-blind).

AMENDMENT-5 (registered before any corrected-baseline run, with the twenty-seed grid already known — disclosed) pre-specifies the corrected-baseline layer: the strong centralized solver of §3, a paired TOST equivalence test at margin ± 0.02 F1 (equivalence iff the 90% t -CI over the twenty paired per-seed $\Delta F1$ lies entirely inside the margin — Schuirmann at $\alpha=.05$), exact Wilcoxon + Holm over the four split×block comparisons, and a FedProx $\mu \in \{.01, .1, 1\}$ sweep restricted to the primary split (descriptive). **Pilot limits.** simulation-only FL; two ecosystems (no Maven); popularity-derived benign labels; hand-curated API mapping tables; the registered twenty-seed protocol makes seed-level tests attainable, but the design remains pilot-scale in corpus and client count.

5 RESULTS

Federation is equivalent to a strong centralized baseline on the primary split (pre-registered TOST). Under the corrected baseline (Table 1), FedAvg matches the strong centralized solver on the primary group split for both blocks: graph $\Delta F1 - .0065 \pm .0171$, 90% CI $[-.0133, +.0003]$; hashing $\Delta F1 - .0024 \pm .0117$, $[-.0070, +.0022]$. Both CIs lie entirely inside the pre-registered ± 0.02 equivalence margin: the TOST equivalence test **passes** on the primary split (exact Wilcoxon .119/.312, Holm .356/.624). This upgrades the earlier “statistically indistinguishable” observation to a formal negative claim: at twenty paired seeds, federation has no measurable F1 cost on the leakage-resistant split, and any undetected cost is bounded by the registered margin.

Retracted claim: “text degrades under federation” was a baseline artifact. Under the earlier protocol we reported a *confirmatory* FL cost on TF-IDF text ($\Delta F1 - .0521 \pm .0219$, exact $p=3.8 \times 10^{-6}$, Holm 1.1×10^{-5}) with a degenerate all-malicious FedAvg mode (recall = 1.0 in 79/80 ecosystem cells; the minority client collapsed under an unweighted average: group-split pypi F1 .671±.059 vs. .790±.076 centralized). That claim does not survive the corrected baseline, and we retract it. Two defects explain it entirely: (i) the TF-IDF featurizer fitted its vocabulary on the pooled corpus — a centralized step unavailable to real federated clients — and degenerated under the weight average; (ii) the earlier solvers were undertrained. With a stateless hashing featurizer and converged solvers on both sides, the degradation vanishes: group $\Delta F1 - .0024$ (n.s., Holm .624), random $-.0037$ (n.s., Holm .624); the degenerate mode is gone (0/360 corrected rows with recall $\geq .999$; hashing FedAvg reaches .906±.020 F1 on the random split with real between-seed variance). Under the corrected protocol, text and graph features are both federation-robust.

Centralized wins only on the leakage-prone secondary split (disclosed). On the random split with graph features — the secondary split that shares a package between train and test for 35% of test rows — the strong centralized baseline beats FedAvg by .016 F1 on average ($\Delta F1 - .0160$, sign 4/15/1, exact $p=.0062$, Holm $p=.025$; the TOST CI $[-.0251, -.0070]$ lies entirely below zero — a directional difference, not missing power). The honest summary: with a properly converged centralized baseline, federation carries no measurable cost on the primary leakage-resistant split, and a

Table 1: Corrected-baseline grid (AMENDMENT-5, registered before the run): twenty seeds, F1/AUC mean $\pm$ std over seeds (population, $ddof=0$). Strong centralized = lbfgs LR on train-fit standardized features (C by train-only 3-fold CV); FedAvg = converged local fit + size-weighted average (a fixed point from round 2 in 80/80 cells); per-client-best is oracle per-sample routing (an upper bound, not a deployable system). Text block = stateless HashingVectorizer (2^{18} , L_2). FedProx (routing fixed) stays within $\pm.005$ F1 of FedAvg at every $\mu \in \{.01, .1, 1\}$ on the primary split (all n.s.; §5). 360 runs, mock=false, config 5ee3764c59a8e4b3.

Run	group (PRIMARY)		random (secondary)	
	F1	AUC	F1	AUC
graph strong centralized	.878 $\pm$.042	.890 $\pm$.040	.887 $\pm$.020	.899 $\pm$.022
graph FedAvg	.871 $\pm$.043	.879 $\pm$.043	.871 $\pm$.029	.893 $\pm$.025
graph per-client-best	.861 $\pm$.041	.887 $\pm$.040	.875 $\pm$.024	.899 $\pm$.021
hashing strong centralized	.875 $\pm$.039	.896 $\pm$.036	.910 $\pm$.020	.939 $\pm$.019
hashing FedAvg	.872 $\pm$.036	.909 $\pm$.034	.906 $\pm$.020	.952 $\pm$.015
hashing per-client-best	.884 $\pm$.031	.916 $\pm$.034	.915 $\pm$.026	.952 $\pm$.021

Table 2: Corrected-baseline comparison (FedAvg – strong centralized), paired per seed over 20 seeds, F1. TOST: pre-registered equivalence margin $\pm.02$ – equivalence holds iff the 90% t -CI of the mean $\Delta F1$ lies entirely inside the margin (Schuirmann at $\alpha=.05$). Wilcoxon = two-sided exact signed-rank; p^{Holm} = Holm-adjusted over the four split $\times$ block comparisons. PASS/FAIL is the TOST verdict, not a significance claim: the random/graph FAIL is a CI entirely *below* zero (a directional centralized win on the leakage-prone split).

Split / block	$\Delta F1$ mean $\pm$ std	$n_+/n_-/n_0$	90% CI	TOST ($\pm.02$)	Wilcoxon p (p^{Holm})
group / graph	-.0065 $\pm$.0171	6/12/2	[-.0133, +.0003]	PASS	.119 (.356)
group / hashing	-.0024 $\pm$.0117	7/13/0	[-.0070, +.0022]	PASS	.312 (.624)
random / graph	-.0160 $\pm$.0228	4/15/1	[-.0251, -.0070]	FAIL	.0062 (.025)
random / hashing	-.0037 $\pm$.0135	5/12/3	[-.0091, +.0017]	PASS	.404 (.624)

small cost only under the leakage-prone random split. “FL is never worse than centralized” is *not* claimed.

The earlier centralized collapse was undertraining, not data structure. The earlier grid showed *centralized* training collapsing in 2/160 seed-level cells (F1 .375 and .326) while FL stayed $\geq .73$; we reported this as central-side instability. The corrected baseline re-attributes it: the same convex problem on the same splits, solved to convergence on standardized features, collapses in 0/80 cells (max $n_{\text{iter}}=69 \ll 5000$; minimum cell F1 .769), and the random-split centralized mean moves .835 $\pm$.113 $\rightarrow$.887 $\pm$.020. Single-run conclusions can err in both directions – the multi-seed lesson stands, but the instability belonged to the solver, not to centralized training.

FedProx (routing fixed): separates in probabilities, not in F1. With the routing bug fixed, the corrected FedProx solver differs from FedAvg at the probability level from $\mu=.01$ (max $|\Delta p|=.0022$ on the audited real cell; unit tests pin $\mu=1$ differing and $\mu=10^{-9}$ coinciding with FedAvg), but no μ separates significantly at the metric level on the primary split: graph $\Delta F1(\text{FedProx}-\text{FedAvg}) = +.0044 \pm .0140$ ($p=.084$), $+.0048 \pm .0214$ ($p=.452$), $+.0043 \pm .0301$ ($p=.475$); hashing stays within $\pm.004$ F1 of FedAvg at every μ (all $p>.21$). FedProx corrects the optimization but does not change the F1 conclusion at this scale; the sweep is descriptive (registered as such, no multiplicity correction). **Stabilized-centralized arm (registered; run once; single config).** The registered stabilization protocol – per-cell lr sweep $\{0.01, 0.03, 0.1\}$ with train-only 3-fold-CV early stopping (patience

3, max 30 epochs; retrain on the full train split, eval on the untouched global test) – ran over 20 seeds $\times$ {group,random} $\times$ {graph,tfidf}: 80/80 real cells under one configuration (sha16 e0cd260b014556bb), 240 sweep entries with 0 undefined fold evals. The sweep selects lr 0.1 in 58/80 cells, 0.03 in 21, 0.01 in 1; mean best epoch 12.05 (min 2, max 30). Outcome: on graph features the arm is F1-neutral (early – fixed $\Delta F1$ +.0011 group, -.0015 random; group/graph .859 $\pm$.046 early vs. .858 $\pm$.047 fixed). On tfidf it loses (-.0511 group, -.0561 random), and the early-stopped tfidf arm predicts positive on $\approx 99.5\%/99.6\%$ of test rows (group/random): CV-AUC-based selection parks the tfidf arm at a near-all-positive operating point, so the tfidf $\Delta F1$ is block-specific operating-point behavior, *not* a general verdict on early stopping (the calibration analysis below resolves this: no all-malicious mode exists under the fixed configuration). Three scope limits travel with these numbers: early-vs-fixed is an aggregate difference only (15/1/64 cells better/tie/worse; no paired test or CI yet); all 80 cells are the ecosystem partition (single-partition scope); and the random/graph fixed baseline is outlier-driven (F1 std .1133 fixed vs. .0356 early-stopped). The question this arm left queued – re-testing FedAvg-vs-centralized under a stabilized baseline – is now answered by the AMENDMENT-5 corrected protocol above (TOST equivalence PASS on the primary split); the arm remains aggregate-only and is retained as solver-instability evidence superseded by the corrected baseline.

Refreshed grid and per-sample probabilities (independent refresh audit; earlier protocol). The full earlier-protocol

grid was re-run under the original registered configuration (same e0cd260b014556bb) and re-audited: 640/640 rows real (mock=False) over 20 seeds $\times$ 8 (partition $\times$ split $\times$ feature-block) cells $\times$ 4 methods (160 cell-seeds each), covering both the two-client ecosystem partition and the three-client npm_hook partition; the centralized ecosystem means reproduce the originally registered fixed baselines exactly (e.g., group/graph .858 $\pm$.047). This grid is retained as the audit substrate for the legacy numbers quoted above and for the calibration analyses below — all federation *claims* now rest on the corrected baseline. Per-sample test probabilities for all 160 centralized cells are released (probs_dump.jsonl: 1 meta + 19,484 real rows, 12,680 malicious / 6,804 benign; per-cell predicted-positive rate .1417–.9929, tfidf .6847–.9000; 342/19,484 probabilities are exactly 0 or 1, all in the graph block) — trained at the config lr/rounds, i.e. not the stabilized early-stopping arm — and are the input for the calibration analysis below. Two audit risks travel with the refresh: the ecosystem random/tfidf FedAvg cell is *seed-invariant* (F1 .784 bit-identical across all 20 seeds, std .0000 — consistent with the tfidf near-degenerate operating point retracted above; variance and paired statistics on that cell are not meaningful), and the per-seed inferential layer was flagged *stale* at refresh time (the grid pipeline summary self-labeled $n=5$ (AMENDMENT-3) while listing twenty seeds); that flag is now *resolved*: scripts/gen_paper_numbers.py recomputes the inferential layer from the stored per-seed deltas of the refreshed grid and reproduces every quoted earlier-protocol value (Holm random/graph corrected .266 $\rightarrow$.265), while the three-client fallback comparison remains aggregate-only.

Calibration, oracle thresholds, and the class-weight arm (independent calibration audit). On the released per-sample test probabilities of the 160 centralized cells — plus a weighted-BCE retrain of the same cells (pos_weight = $n_{\text{neg}}/n_{\text{pos}}$ on train, same lr/epochs/seed protocol; helper-local trainer, disclosed) — three findings, all recomputed by the auditor from the raw dumps. (i) *No all-malicious mode*: 0/40 cells per block $\times$ split are all-malicious at threshold 0.5, so the earlier all-positive parking is specific to the early-stopping protocol; the fixed centralized TF-IDF model’s deficit at 0.5 is calibration and threshold bias, not a trivial classifier (predicted-positive rate@0.5: .692/.719 graph, .803/.806 TF-IDF, group/random). Calibration is still poor for TF-IDF: pooled 15-bin ECE .1891/.2240 (group/random) vs. .0726/.0870 for graphs; Brier .1664/.1574 vs. .1478/.1659 — and the ECE column is the *unweighted* model’s; the weighted arm’s calibration is not reported. (ii) *Threshold sweep — a test-set oracle, not a deployable result*: thresholds are selected on the same pooled test rows they are scored on (no validation split in the analysis script), so these gains are post-hoc upper bounds: TF-IDF improves .8466 $\rightarrow$.8790 (thr .55, group) and .8432 $\rightarrow$.8955 (thr .60, random), while graphs are best at the default .50 (no gain: .8618/.8416). A validation-locked re-evaluation is queued before any tuned-threshold claim is made; until then, the fixed-threshold comparisons above (earlier-protocol centralized cells, quoted above) must not be read as intrinsic model ranking — part of the graph-vs-TF-IDF F1 gap at 0.5 is threshold miscalibration. (iii) *Class weighting is a negative intervention, not a remedy*: weighted-BCE retraining lowers F1@0.5 in 3 of 4 block $\times$ split groups (graph .8618 $\rightarrow$.8347 group, .8416 $\rightarrow$.8278 random;

TF-IDF .8466 $\rightarrow$.8005 group) and helps only TF-IDF/random (.8432 $\rightarrow$.8706) — which still loses to the zero-cost (oracle) threshold sweep on the unweighted model (.8955). Post-hoc threshold adjustment dominates costlier weighted retraining in this setting.

Three-client fallback arm. A registered fallback partition (npm split by install-hook presence: npm_core 214 / npm_hook 114 / pypi 146; the hook client is 99.1% malicious — the corpus has exactly one hooked benign package, which lands in train in all five group splits) differs negligibly from the two-client partition (group/graph $\Delta F1 +.024 \pm .120$, exact Wilcoxon $p=1.0$). This is a same-corpus partition, *not* cross-ecosystem FL: the third-ecosystem source probe failed and no ingestible source was found (disclosed).

Graph vs. text is split-dependent, honestly. Under the corrected baseline the ranking picture changes with the split (Table 1): on the primary split the blocks differ by at most .030 AUC (graph FedAvg .879 $\pm$.043 vs. hashing FedAvg .909 $\pm$.034; centralized .890 vs. .896), while on the random split the text block ranks higher (hashing FedAvg .952 $\pm$.015 vs. graph .893 $\pm$.025). The earlier protocol’s text miscalibration at the fixed threshold (recall ≈ 1) was the baseline artifact retracted above, not a property of text features. We do not claim graph superiority; the claim is complementarity under calibration-sensitive metrics and interpretability (named behavior nodes), at the cost of coverage (Table 3). Ranking-quality extensions (AUC-PR, ECE and reliability curves, train-CV threshold selection) are specified in the project roadmap, not yet run.

Trivial-structure baseline: the marginal value of behavior graphs is real but bounded. A metadata-only baseline using five trivial features (file count, parse failures, entry-point flags, empty-graph indicator) already reaches F1 .82–.86 (cell means .823–.855 across both splits and both corpus variants; twenty seeds, both arms under the same standardized sklearn protocol), showing that most detectable signal is cheap metadata. The 18-feature behavior graph — a strict *superset* of these signals (the empty-graph flag is derivable from n_{nodes}) — adds a statistically significant marginal gain on top of it ($\Delta F1 +.0227 \pm .0193$ group / $+.0487 \pm .0163$ random; exact Wilcoxon over twenty seeds $p=1.9 \times 10^{-5}$ and $p=1.9 \times 10^{-6}$; per-seed exact McNemar pooled discordants 171/83 and 236/68 favoring the graph). The advantage persists on the 500 non-empty-graph rows (group $\Delta F1 +.0322$, $p=.0012$; random $+.0550$, $p=1.9 \times 10^{-6}$), so it is not an artifact of the empty-graph flag. Graph features therefore beat a trivial structural baseline significantly — this is not shortcut learning — but the trivial baseline’s own strength bounds the claim: its F1 of .823–.855 means the marginal value of behavior graphs is real yet bounded; we position them as a refinement of, not a replacement for, metadata signals. One disclosure: a sixth checklist feature (byte size) is absent from the extracted schema and was neither imputed nor substituted.

Sensitivity arms (single seed 20260922). DP ($\sigma=0.1$, $\epsilon \approx 48.4$ per round, single-query bound): F1 0.915 (−0.8 points) — privacy noise is affordable at this scale, though the R-round composition is unproven. MLP matches LR exactly on the primary metrics. Dropping the 103 empty-graph rows moves central F1 to 0.821 ($n=500$): the all-zero rows are not harmless artifacts but carry ecosystem signal (disclosed, not exploited).

Table 3: Coverage after the file-selection fix (v2). The v1 selector flooded its 12-file budget with `__init__.py` and metadata files, manufacturing 16 spurious empty pypi-benign graphs (audit finding); pypi-benign coverage rises 55.6% $\rightarrow$ 70.0%.

Cell	with-graph	coverage
npm benign	105/123	85.4%
npm malicious	252/298	84.6%
pypi benign	63/90	70.0%
pypi malicious	80/92	87.0%
total	500/603	82.9%

Table 4: KB ablation on the primary (group) split, initial run under KB v2 (54.7% type coverage); the completed KB v3 (100%) re-run is reported in the text. “on” appends `kb_risk_ratio` / `kb_confidence` / `kb_unsure_ratio` to the 18 graph features.

Run	F1		AUC	
	off	on	off	on
FedAvg	.923	.922	.916	.911
centralized	.928	.938	.926	.932
pypi F1 (central)	.875		.966	

5.1 Knowledge-base ablation

The graph universe of the 603-sample corpus contains 137 unique API names; 122 were unknown to the 20-entry seed KB. An initial batch classified the top 60 by document frequency with local Qwen2.5-Coder-3B-Instruct through the refusal-gated builder: **0 refusals, 0 retries, 0 UNSURE** (the 0.5B pilot model produced 8/10 UNSURE on the same protocol — model scale matters), at ~ 15 s/call on MPS; the remaining 62 tail APIs were counted as unsure by construction (disclosed). A follow-up batch completed the universe: the KB now covers **137/137 API types and 2,299/2,299 API instances** (100.0%; 142 entries = 20 seed + 122LLM-classified, zero unsure), raising mean `kb_confidence` over the graphs from .878 to .907 — with the disclosure that 113/122LLM entries self-report confidence clipped at 0.90 (a weak calibration signal). The FL ablation below was run under the earlier 54.7%-coverage KB; a registered re-run under the complete KB needs no LLM calls (KB features are a pure join) and is reported below.

Under the initial KB-v2 run, KB features were neutral under FedAvg (-0.1 pt F1) and added $+1.0$ pt F1 / $+0.6$ pt AUC to centralized training, with the gain concentrated on the minority client (pypi F1 .875 $\rightarrow$.966; the npm subgroup slips .940 $\rightarrow$.932; subgroup $n=36$ — indicative, not significant; reported as an effect size, not a test). The robust yield is explainability, not accuracy: every API call in a verdict now carries a gated, cached justification (semantic class + rationale) covering the full API universe — the contribution of the KB is this audit trail, not classifier performance.

KB-v3 pure-join re-run (independently audited; single seed 20260922, graph features only). The complete-KB re-run needs no LLM calls — KB features are a pure dict-join (verified: no network/LLM path) — and joins the full corpus API universe:

instance coverage $2,299/2,299 = 1.0$, mean unsure ratio 0.0, over 142 KB entries (20 seed + 122 LLM-classified, 0 unsure; 5 entries cover APIs absent from this 603-record corpus). At the shared seed the centralized gain matches the initial run’s magnitude ($+0.0103$ group, $+0.0112$ random F1), but the FedAvg effects are of *mixed sign* (-0.0009 group, -0.0224 random). With $n=1$ seed, all four deltas sit far below the measured per-cell seed variability (centralized group/graph F1 std .0468): no statistical claim is possible, and the honest reading is a *backend-dependent interaction* — small positive on centralized, negative under FedAvg — pending the twenty-seed inference layer and a KB-entry ablation (seed-only vs. LLM-only vs. coverage-matched control). Denominator note: “mean confidence” is 0.75215 instance-weighted over the 2,299 joined API instances but 0.8993 entry-level over the 142 KB entries — both correct, not interchangeable.

5.2 Safety layer on real packages

Blocking does not replicate at 2–3B. RR = 0.000 across all 360 generations (exact McNemar vs. P0: $p=1.0$ everywhere): inviting either model to analyze a credential-stealer through the defensive static-review frame never triggered a single refusal, in any arm. The measured scope of this null across the project is 2,700 defensive-task generations with no overt refusal (vulnerability-domain batches 180 arms + 900 E2/E3 conditions; 1,200 in the expanded replication; package-domain 360 + 60) under the evaluated <4 B models and prompt families; in the same runs the deliberately harmful OR-Bench probe prompts were refused 125/225 times, so the refusal pathway is functional — the defensive task itself never triggers it. Directive-refusal bias is *not supported* at 2–3B for the tested prompt families. We deliberately do not generalize this to “safety alignment does not transfer” — the claim is closed only at these scales, families, and prompts.

The corruption channel re-types as recall-perturbing. All 11 benign $\rightarrow$ malicious flips across arms land on genuinely malicious packages (checked per sample), so the pre-registered corruption rule (≥ 1 flip) fires as written but measures a *sensitivity shift on malicious packages*, not the wrongful flagging of benign code. The direction is model-dependent — undirected: granite-3.3-2B recall .533 $\rightarrow$.800 under P2, while Llama-3.2-3B falls .133 $\rightarrow$.033 (four detections lost, one gained — a measured cost). Benign false positives are 0/180 across all arms and models (0/60 pooled-P2 benign-records, bounding the FP rate below $\approx 5\%$ by rule of three). Context thus shifts sensitivity on malicious packages in both directions without touching benign false positives — a *recall-perturbing* channel, unlike the vulnerability domain’s false-positive channel; the initial-batch numbers (“FP-bias” .2 $\rightarrow$.6, granite 1.0) must be re-read as recall on the 10 malicious packages (2/10 $\rightarrow$ 3/10 $\rightarrow$ 6/10; granite 5/5) with benign FP 0/30.

Safety expansion to $n=100$ (follow-up expansion). The merged batch is 360 verbatim copies of the $n=60$ records plus 240 new generations over 100 distinct samples (50 malicious / 50 benign; the new 40 are gate-passers [30:50] of the same seeded draw — asserted zero overlap); copied records carry no mock field (mock=None), all 240 new rows are mock=False; of the new generations, 192 are fresh (MPS, bfloat16) and 48 are deterministic-cache hits (temperature 0, seed 1234). Pooled: malicious recall

Table 5: Safety arms on 60 real packages, per model (30 malicious + 30 benign; 60 paired generations per model/arm). RR = refusal rate; malicious recall = TP-rate on the 30 malicious packages; FP benign = benign false-positive rate. *Semantics note:* the initial table labelled this quantity “FP-bias” (.2/.3/.6, granite 1.0); a subsequent audit showed the then-code-path computed the recall on *malicious* packages — those numbers must be re-read accordingly (metrics_version 2; raw records unchanged).

Model / arm	RR	flips b→m / m→b	recall (P0→arm)	FP benign
llama P0 neutral	.00	– / –	.133	0/30
llama P1 offensive	.00	0 / 0	.133 → .133	0/30
llama P2 advisory	.00	1 / 4	.133 → .033	0/30
granite P0 neutral	.00	– / –	.533	0/30
granite P1 offensive	.00	2 / 0	.533 → .600	0/30
granite P2 advisory	.00	8 / 0	.533 → .800	0/30

.350 → .380 → .440 (P0→P1→P2), benign FP 0/0/2-of-100, parsed 1.0/.995/1.0, RR 0.0 everywhere with zero discordant pairs (exact $p=1.0$); verdict flips 3 / 0 at P1 (199pairs) and 19 / 8 at P2 (200 pairs); pre-registered verdicts blocking_supported=false, corruption_supported=true (P1, P2). Two interpretation rules, stated with the numbers. (i) *The blocking column is structurally uninformative here:* RR $\equiv 0.0$ in all arms including P0 (0 refusals in 600 records) — the models never block this batch, so $p=1.0$ is not evidence that “blocking has no effect”; the finding is structural blocking-absence, not a treatment null. (ii) *Pooled numbers mask opposite per-model movement:* at P2, llama’s recall collapses .160 → .060 (all 8 malicious→benign flips are llama’s; benign FP 0) while granite’s recall rises .540 → .820 but incurs the pooled FP (0 → .040; both pooled FP records are granite’s) — the pooled corruption verdict is rule-driven (min-flip-count ≥ 1) and its P1 instance rests on just 3 flips; the granite-vs-llama divergent corruption response is itself the salient observation. Selection caveat: this run’s scan covered 265 records (100 gate-pass); the plan-level pool figure (560 → 230; 147 mal/83 ben) remains unverified.

Mechanism not yet isolated. We cannot yet distinguish “the advisory content helps the model reason about malicious behavior” from “any unusual in-package content perturbs the verdict”; the recall *increase* is therefore not claimed as a benefit, and the *decrease* (llama) is the measured harm. The matched-scale family contrast (granite +8 vs. llama –3 net flips at P2) shows the effect is family-specific at 2–3B. **Mechanism ablation (independent audit; null with low power).** The registered ablation is now run — same-meaning paraphrase and word-scrambled controls of the neutral task over 30 malicious gate-passers of the pilot seeded draw (zero new selection): 180 real rows (all mock=False; 90 per model, 60 per variant), parsed rate 1.000 (179 ANSWER + 1 PARTIAL, counted parsed by the parsed-JSON criterion), refusal 0.000 in every cell. Neither variant significantly moves the malicious-verdict rate (exact McNemar, 30 pairs per cell): granite .6000 → .6667 (paraphrase, $p=.5$), → .6333 (scrambled, $p=1.0$); llama .1333 → .2333 ($p=.25$), → .1667 ($p=1.0$). Three caveats travel with this null: (i) *power* — at $n=30$ with every $p \geq .25$, “no effect” is indistinguishable from a small effect; this is a suggestive null-with-low-power, not an established null (the uniformly non-negative direction is suggestive only); (ii) *provenance* — 96/180 rows are deterministic-cache hits (temperature 0, seed 1234, byte-identical prompts: all 60 verbatim rows plus 9 paraphrase + 9 scrambled per model; generation dates Sep 21–23), 84 generated fresh — methodologically valid reuse, not

new generation; (iii) *no cross-round comparison* — the granite verbatim rate .60 here is a different sample set (gate-passers [0:30] of the seeded draw) from the ladder’s A0 recall .7627. The directional bookkeeping is corrected from the runner’s first report: granite/scrambled contains exactly one 1→0 flip alongside two 0→1 (net +1); the other three cells are purely 0→1. The FP-directed arm (an advisory asserting “this code is clean”) remains queued. All of this remains pilot-scale: it establishes measurability, the zero-refusal blocking outcome, and direction — not effect size.

5.3 Can sanitization neutralize the channel?

The pairing closes the loop the expansion left open: P2 showed that untrusted in-package content shifts detection sensitivity (§5.2), so a defender’s first move is to remove that content. Defense **D1** (AMENDMENT-6, registered before any round-12 generation) strips comments and Python docstrings from the package file before analysis — string literals are never removed — under an AST-equivalence gate: the stripped file must re-parse to a canonical AST signature identical to the original’s, where the signature prunes exactly the node classes D1 removes; a gate failure excludes the sample, disclosed rather than silently dropped. Removal is line-aware, so the prepended one-line advisory strips back to the byte-identical original file; and since the P0 and P2 (undefended) arms of this round are verified cache hits of the expansion batch, the defended arm is exactly “attack removed,” not a rephrasing: on all 38 gate-passing samples the stripped attack file equals the stripped original byte-for-byte (so $P2+D1 \equiv P0+D1$ at the byte level), with byte-identical $P2+D1$ vs. $P0$ prompts — and temperature-0 completions — for the 10 comment-free originals. The registered hypotheses are deliberately two-directional: H-D1 (restoration) holds where the defense lifts recall back to at least the neutral level within .05; H-D2 (neutralization) where $|\text{recall}(P2+D1) - \text{recall}(P0)| \leq .10$; per-model pass/fail is reported either way, and because one model’s recall *rose* under the attack, the defense-cost check (does stripping the advisory lose the extra detections?) is registered alongside — restore-gain and restore-loss flips are both counted and neither is folded into an “other” bucket. The measured subsets are the seeded draw’s gate-passers (registered 30 malicious + 15 benign; realized 24 malicious + 14 benign per model after 7 of the 45 drawn snippets — 6 malicious, 1 benign — are comment-only, strip to empty, and are excluded and disclosed per the A6.1 gate; exclusion is label-independent, Fisher $p = .40$; coarse single-sample resolution, disclosed), with exact

Table 6: Attack–defense pairing (P2 advisory vs. the D1 comment/docstring strip, AST-equivalence gated). Recall on the malicious subset; $+g/-l$ = restore-gain (0→1) / restore-loss (1→0) flips of D1 vs. the attack arm; p = exact paired McNemar (P2 vs P2+D1); FP = benign false positives (attack → defended). Realized subsets after the A6.1 gate: 24 malicious + 14 benign per model (7 of the 45 drawn snippets are comment-only, strip to empty, and are excluded and disclosed — 6 malicious, 1 benign; Fisher p = .40, no label bias). P0 and P2 arms are verified cache hits of the $n=100$ expansion; the attack-arm FP column is quoted from that expansion (the r12 batch has no P2 benign arm). One granite P2+D1 output was unparseable (disclosed; baseline-positive). Granite P0 vs. P2+D1 has 0 discordant pairs (23 parsed; exact upper 95% bound on the discordant rate $\approx 12\%$), so every reported p is descriptive.

Model	rec P0	rec P2	rec P2+D1	$+g/-l$	p	FP
llama-3.2-3B	.167	.042	.042	1/1	1.0	0/50→0/14
granite-3.3-2B	.667	.833	.652	0/4	.125	2/50→0/14

paired McNemar for (P2 vs P2+D1) and (P0 vs P2+D1), benign false positives per arm, and refusals — as everywhere in this section — never mapped to a verdict. Table 6 reports the pairing over 76 paired real packages (38 gate-passing samples $\times$ two models; realized 24 malicious + 14 benign per model; 47 parsed malicious pairs pooled; mock=false): pooled attack recall .438 (21/48) vs. defended .340 (16/47; one granite P2+D1 output unparseable, disclosed), neutral .417 (20/48); restore-gain 1, restore-loss 5 (exact McNemar p = .219, $n=47$, 1 vs. 5 discordant, against the attack arm; p = .375, 1 vs. 4, against the neutral arm); benign FP 0/28 defended (the attack arm carries no benign rows in this batch; at the $n=100$ expansion the advisory caused 2/100 benign FPs, all granite — quoted from that expansion, not re-measured here). Every pooled statistic is descriptive: the pool adds opposite per-model movements (loss 5 vs. gain 1) and no paired comparison in this round reaches significance.

Per-model outcome. On granite-3.3-2B, where the advisory had *raised* recall (.667 $\rightarrow$.833) while inducing four benign→malicious verdict flips, D1 fully neutralizes the advisory channel: P2+D1 verdicts are identical to the neutral baseline on every parseable pair (23/23; the single unparseable P2+D1 output was baseline-positive) and all four attack-induced flips revert, at zero benign FP (0/14) — H-D1 and H-D2 pass on this model. On Llama-3.2-3B, where the advisory had *suppressed* recall (.167 $\rightarrow$.042), D1 does not restore detections (.042): the four lost detections are byte-identical 2500-character snippets from four archive versions of a single package family (19 distinct families in the subset) whose own comment the defense also strips — a defense cost we pre-registered as a possible outcome — and the one detection the advisory itself had supplied is lost again (the single P2+D1 detection is yet another family, not a recovered one; at family granularity llama detects 1/19 in every condition). Pooled contrasts are therefore descriptive throughout, and the llama restoration failure is a one-family observation at this draw.

6 DISCUSSION

When graphs beat, and lose to, cheap signals. Under the corrected baseline, raw text ranks higher on the leakage-prone random split while graph features are within noise of it on the primary split (Table 1); and against a trivial metadata baseline, graphs add a significant but bounded margin (+.02–.05 F1, \$5) — a refinement, not a replacement. The honest conclusion is that the behavior-graph hypothesis is *not yet* validated as strictly better; its value is calibration, interpretability, and coverage robustness under the group split. **Boundary condition (family shift).** The group split removes version-level leakage within a package (AMENDMENT-1) but not whole-family shift — unseen families sharing an actor or lineage. The reported AUC ordering is conditional on that threat model; a registered leave-families-out comparison (graph vs. TF-IDF degradation under family shift) must decide any robustness claim, and temporal evaluation on fresher attacks is out of scope for this pilot. **Retracted findings.** Our interim run on the v1 features showed a dramatic FedAvg minority-client harm (pypi F1 0.57 vs. 0.82 centralized). After the file-selection fix the harm vanishes (registered grid: group-split pypi F1 .851 $\pm$.080 under FedAvg vs. .830 $\pm$.100 centralized across seeds — if anything reversed). We report this because the “FedAvg harms minorities” narrative was an artifact of a feature bug — a caution against interpreting FL failures before auditing the features. The corrected-baseline layer adds two more entries in the same genre: the “text degrades under federation” claim was an artifact of pooled-vocabulary fitting plus an undertrained solver, and the apparent centralized-training collapse belonged to the solver, not to centralized training (§5). Single-run conclusions can err in both directions — which is why every federation claim in this paper now rests on a pre-registered, converged, twenty-seed comparison (TOST equivalence on the primary split; the small centralized win on the leakage-prone split disclosed). **Safety transfer.** The prior study’s *machinery* (refusal monitor, AST-gated advisory injection, frozen rules) ports mechanically to packages, but the corruption channel does not port as the same *phenomenon type*: in the vulnerability domain untrusted context created false positives on benign code; in the package domain it perturbs recall on malicious packages with benign FP = 0 (§5.2). P2’s advisory-in-package attack is a package-specific threat the vulnerability study could not express, and the AST gate makes it safely measurable; because the direction is model-dependent — the $n=100$ expansion confirms opposite per-model signs (§5.2) — follow-up arms must pre-register *both* outcome directions (harm and benefit) before running. **Scale versus model family (confound narrowed by a post-hoc granite ladder).** Across the project, Qwen2.5-Coder is inert at 3B and 7B (the pre-registered 7B harm-absent verdict is SUPPORTED), Llama-3.2-3B shows defense harm at 3B, and granite-3.3-2B moves in the opposite direction in the package domain. A post-hoc granite-3.3-2B re-run of the vulnerability-domain ladder (240 cache-only generations, zero new decoding, refusal rate 0.0 in every arm) now shows the reassertion harm replicates at 2B: recall on vulnerable files .7627 $\rightarrow$.0667 under rung A5 (41/0 flips; McNemar continuity-corrected χ^2 $p=4.2\times 10^{-10}$, exact binomial 9.1×10^{-13} ; bootstrap Δ CI [−.81, −.58]) and .7627 $\rightarrow$.4000 under the milder advisory-only rung A1 (23/2 flips,

$p=6.3\times 10^{-5}$) — the pre-registered “A1-minimal-safe” hypothesis is REFUTED (harm attributable to A1), the harm-absent hypothesis is NOT_EVALUABLE on this model (harm present), and the benign rung adds a verdict false-positive bias ($0/30 \rightarrow 20/30$, exact $p=1.9\times 10^{-6}$). Same model, opposite domain signature: in the package domain granite’s advisory arm moved malicious recall *up* with zero benign FP, while in the vulnerability domain it collapses recall and manufactures benign FPs — the effect is family-and-domain specific rather than a pure scale effect, and every conclusion remains closed at 2–3B. Caveats stated with the numbers: (i) the A5 arm is cache-only and its prompt hashes match none of the cache’s stored keys (0/60, vs. 60/60 for A0/A1/benign), so “byte-identical prompts” for A5 rests on the cache layer’s keying and is not independently verifiable (a one-time GPU regeneration of A5 would close this gap; queued); (ii) the metrics file mixes McNemar variants — the two quoted vulnerability p values are continuity-corrected χ^2 and the file’s exact binomial values are smaller still (9.1×10^{-13} , 1.9×10^{-5}), so the direction of every conclusion is unchanged but the method should be unified (queued); (iii) A0’s recall denominator is 59 parsed records (1 unparseable) vs. 60 for A1/A5, and the paired tests use the 59 valid pairs. **8B feasibility disclosure.** The correct instrument for the scale question (Llama-3.1-8B-Instruct, same family, $\sim 2.7\times$) is infeasible on this stack: four independent MPS load failures (a bf16 harness stall ≥ 27 min in weight copy with zero tokens generated; two fp16 accelerate-dispatch SIGSEGVs — pointer-authentication failures with stacks in `mps::copy_cast_kernel`; a direct fp16 run hung for 8 min until the watchdog killed it), while a CPU proof loaded the sha256-verified weights and generated real tokens at **1.11 tok/s**, making the pre-registered 240-generation $\times$ 512-token ladder ≈ 31 h of decoding alone (NOT_FEASIBLE-in-time). Peak-RSS figures observed live during the failures were not preserved in raw artifacts and are not quoted as evidence; a 3B control validates the load path only (its generation step errored, so it is load-only evidence). Any ≥ 8 B result requires a different runtime (MLX 4-bit or GGUF/llama.cpp — neither present in this environment) and must not share feasibility claims with the transformers/MPS stack. **Threats to validity of the corrected-baseline layer.** Three disclosed artifacts travel with the correction itself. (i) *FedProx routing bug*: in the earlier grid, μ was computed per algorithm but never passed down to the clients, so both arms silently ran FedProx($\mu=.01$); the earlier “FedAvg” rows are re-labeled accordingly (they remain valid as FedProx(.01) runs under the earlier protocol). The fix routes μ per algorithm and is pinned by unit tests ($\mu=1$ provably differs from FedAvg, $\mu\rightarrow 0$ coincides with it, and the audited real cell reproduces its earlier row bit-exactly); a `legacy_mu_routing` flag reproduces the old behavior exactly for reproducibility. (ii) *FedAvg-strong is a converged local-fit-then-average simulation* (a fixed point from round 2 in 80/80 cells), registered as such in AMENDMENT-5 — it does not model partially updated local SGD work. (iii) *Shared preprocessing*: federated clients use the train-fit scaler and the CV-selected C (a central hyperparameter-coordination assumption; training rows only — no test row enters any fit). **External validity.** All results are measured on one wild-capture malicious corpus plus popularity-derived benign labels; external validation on independently collected corpora (e.g., campaign-level industry

feeds, license not yet probed) is required before any deployment claim, and a deployed-data evaluation of the safety arms remains future work.

7 CONCLUSION AND LIMITATIONS

A 603-sample, provenance-complete pilot shows: federated training over ecosystem partitions is *equivalent* to a strong centralized baseline on the primary leakage-resistant split (pre-registered TOST at $\pm .02$ F1: PASS for graph and text features alike), with a small centralized advantage confined to the leakage-prone random split; an earlier text-degradation claim was retracted as a baseline artifact (pooled-vocabulary fitting plus an undertrained solver); representation ranking is split-dependent; behavior graphs add a statistically significant but bounded margin over a trivial meta-data baseline (refinement, not replacement); LLM-built knowledge bases are measurable, with a now complete (100%-coverage) gated API universe whose robust yield is explanation rather than F1; and the safety-layer third threat is real but *recall-perturbing* in this domain, with zero refusals at 2–3B. The first attack–defense pairing gives sanitization a measured answer: the AST-gated strip moves pooled malicious recall $.438 \rightarrow .340$ against the advisory attack (exact McNemar $p = .219$; benign FP 0/28 defended; the attack arm is not re-run in this batch — at the $n=100$ expansion the advisory caused 2/100 benign FPs, all granite) — and the decisive read is again per-model: full neutralization on granite-3.3-2B (verdicts identical to the neutral baseline on all 23 parseable pairs; all four attack-induced flips revert; benign FP 0) versus no restoration on Llama-3.2-3B, whose four lost detections are byte-identical snippets of one package family whose own comment the strip also removes — a defense cost we pre-registered as a possible outcome; every reported p is descriptive at this pilot scale. Limitations: pilot scale (CI widths $\pm .05$ – $.08$ F1), simulation-only trust mechanisms, two languages, popularity-derived benign labels, hand-curated mappings, an LLM-KB whose confidence is self-reported, and a hard scale ceiling: conclusions close at 2–3B, the direction of safety-layer effects follows model family *and* domain (granite-3.3-2B: reassertion harm plus benign verdict bias in the vulnerability domain, opposite-direction recall shift with zero benign FP in the package domain), and ≥ 8 B evaluation requires a different runtime. A Q1-grade system needs: per-seed inference extended beyond the regenerated paired exact tests — effect sizes, bootstrap 95% CIs, and coverage of the stabilized arm (still aggregate-only) across both partitions — a validation-locked threshold evaluation (the pooled-test sweep selects thresholds on the pooled test set — an oracle; the validation-locked rerun is queued), a KB-entry ablation under twenty paired seeds (seed-only vs. LLM-only vs. coverage-matched control; the KB-v3 gain is single-seed), a unified McNemar method and an A5 provenance closure for the granite ladder (one-time GPU regeneration), achieved-power and variance-decomposition analysis (Holm correction is now applied), a real GNN over the graphs, a leave-families-out evaluation, mechanism-isolated safety arms at $n\geq 100$ /model, at least one more ecosystem, audited benign labels, and a deployed-data evaluation of the safety arms. At the close of the present run set, the FedProx implementation audit is complete (routing bug found, fixed, and

unit-pinned) and the primary federation comparison carries pre-registered TOST equivalence; what remains *descriptive in strength* is the stabilized-centralized arm (aggregate-only), the single-seed KB-v3 deltas, and the FedProx μ sweep (registered as descriptive); the twenty-seed paired exact-test layer over the earlier grid is regenerated and script-verified (`scripts/gen_paper_numbers.py`), but until effect sizes with CIs and the stabilized-arm inference land, all early-vs-fixed evidence in this paper remains descriptive in strength.

DATA AND PROVENANCE

Manifest, features, graphs, per-row results with mock flags, audit reports, and the pre-registration amendments are in the anonymized artifact tree released with this submission (dataset manifests under `data/`, all measured outputs under `outputs/` incl. `fl_multiseed/grid_results.json`, `p0/p0_results.json` (the corrected-baseline grid), `trivial/trivial_results.json`, and `safety/safety_metrics_n60.json`, the pre-registration document `docs/preregistration.md` with AMENDMENT-1/-3/-4/-5, and the audit reports directory). Paths are shown relative to the artifact root; per-number provenance is annotated in the source of this manuscript (% SRC: comments), and every number is macro-generated from these artifacts by `scripts/gen_paper_numbers.py` (`p0_macros/numbers.tex`).

REFERENCES

- [1] Anonymous. 2026. Measuring and Mitigating Safety-Layer Interference in LLM Vulnerability Analysis. Anonymous manuscript under double-blind review. Unpublished companion measurement study; no public identifier as of this submission. An earlier bibliography entry mistakenly attached arXiv:2603.01246 (which is Campbell et al., “Defensive Refusal Bias”) — corrected in the round-11 audit.
- [2] David Campbell, Neil Kale, Udari Madhushani Sehwag, Bert Herring, Nick Price, Dan Borges, Alex Levinson, and Christina Q Knight. 2026. Defensive Refusal Bias: How Safety Alignment Fails Cyber Defenders. *arXiv preprint arXiv:2603.01246* (2026). arXiv record verified 2026-09-27 (v2).
- [3] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. arXiv:2002.01139 (arXiv comment: “To appear in NDSS Symposium 2021”); MalOSS collection; authors and venue corrected in round-11 audit.
- [4] Xingnan Gao, Xiaobing Sun, Sicong Cao, Kaifeng Huang, Di Wu, Xiaolei Liu, Xingwei Lin, and Yang Xiang. 2025. MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem. In *34th USENIX Security Symposium (USENIX Security 2025)*. arXiv:2506.14466; venue per USENIX program, arXiv record verified 2026-09-27.
- [5] S. Halder, M. Bewong, A. Mahboubi, Y. Jiang, R. Islam, Z. Islam, R. Ip, E. Ahmed, G. Ramachandran, and A. Babar. 2024. Malicious Package Detection using Metadata Information. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*. <https://doi.org/10.1145/3589334.3645543> System name MeMPtec; arXiv:2402.07444; venue corrected from ASE to WWW in round-11 audit.
- [6] Cheng Huang, Nanman Wang, Ziyan Wang, Siqu Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium (USENIX Security 2024)*. arXiv:2403.08334; arXiv comment confirms USENIX Security 2024 acceptance.
- [7] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (S&P)*. 1509–1526. arXiv:2204.04008; venue verified 2026-09-27.
- [8] Marc Ohm, Henrik Plate, Andreas Sykosch, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. arXiv:2005.09535 (ID corrected in round-11 audit; 2005.09561 is an unrelated paper); BKC dataset.
- [9] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2025. Leveraging Large Language Models to Detect npm Malicious Packages. In *47th IEEE/ACM International Conference on Software Engineering (ICSE 2025)*. arXiv:2403.12196 (v1 title: “Shifting the Lens: Detecting Malware in npm Ecosystem with Large Language Models”).
- [10] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruiqi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Transactions on Software Engineering and Methodology* 34, 4 (2025), 1–28. <https://doi.org/10.1145/3705304> System name Cerebro; arXiv:2309.02637; CrossRef record verified 2026-09-27.
- [11] Peiheng Zhou, Ming Hu, Xingrun Quan, Yawen Peng, Xiaofei Xie, Yanxin Yang, Chengwei Liu, Yueming Wu, and Mingsong Chen. 2024. An Empirical Study of Vulnerability Detection using Federated Learning. *arXiv preprint arXiv:2411.16099* (2024). No journal ref as of 2026-09-27; cited as preprint.